

Unit 3: Hive

Big Data Analytics — Exam Notes

Comprehensive & Exam-Ready

Contents

1	Running Hive	4
1.1	Configuring Hive	4
1.1.1	Key Configuration Files	4
1.1.2	Overriding the Configuration Directory	4
1.1.3	Cluster Connection Details in hive-site.xml	4
1.1.4	Per-Session Configuration using <code>-hiveconf</code>	4
1.1.5	Setting Up Shared Directories (Multi-User Setup)	5
1.1.6	Changing Settings Within a Session — <code>SET</code> Command	5
1.1.7	Configuration Precedence Hierarchy	5
1.2	Execution Engines	5
1.3	Logging	6
1.4	Hive Services	6
1.5	Hive Clients	6
1.5.1	Thrift Client	7
1.5.2	JDBC Driver	7
1.5.3	ODBC Driver	7
2	The Metastore	7
2.1	Hive Architecture	7
2.2	Metastore Configurations	7
2.2.1	1. Embedded Metastore (Default)	7
2.2.2	2. Local Metastore	8
2.2.3	3. Remote Metastore	8
2.3	Important Metastore Configuration Properties	10
3	Comparison with Traditional Databases	10
3.1	Schema on Read vs. Schema on Write	10
3.2	Updates, Transactions, and Indexes	10
3.2.1	Updates and Inserts	10
3.2.2	Transactions	11
3.2.3	Locking	11
3.2.4	Indexes	11
4	SQL-on-Hadoop Alternatives	12
4.1	Cloudera Impala	12
4.2	Hive's Response — Stinger Initiative	12
4.3	Other SQL-on-Hadoop Alternatives	12

5	HiveQL	13
5.1	SQL vs. HiveQL — Feature Comparison	13
6	Data Types	14
6.1	Primitive Data Types	14
6.1.1	Key Notes on Primitive Types	14
6.2	Complex Data Types	15
6.2.1	Complex Type Declaration (Angled Bracket Notation)	15
6.2.2	Field Accessor Operators	15
7	Operators and Functions	15
7.1	Operators	15
7.2	Built-in Functions	16
7.3	Type Conversions	16
8	Tables	17
8.1	Multiple Database / Schema Support	17
8.2	Managed Tables vs. External Tables	17
8.2.1	Managed Table Example	17
8.2.2	External Table Example	18
9	Partitions and Buckets	18
9.1	Partitions	18
9.1.1	Creating and Loading a Partitioned Table	18
9.1.2	Partition Directory Structure (HDFS)	19
9.1.3	Querying Partitions	19
9.2	Buckets	19
10	Buckets	20
10.1	Creating a Bucketed Table	20
10.2	Populating a Bucketed Table	20
10.3	Sampling with TABLESAMPLE	21
11	Storage Formats	21
11.1	SerDe (Serializer-Deserializer)	21
11.2	Default Storage Format — Delimited Text	22
11.3	Binary Storage Formats	22
11.4	Custom SerDe — RegexSerDe	23
11.5	Storage Handlers	24
12	Importing Data	24
12.1	INSERT Statement	24
12.2	Multitable Insert	25
12.3	CREATE TABLE...AS SELECT (CTAS)	25
13	Altering Tables	26
13.1	Rename a Table	26
13.2	Add a New Column	26
13.3	Changing Column Metadata	26

14 Dropping Tables	26
15 Querying Data	27
15.1 Sorting and Aggregating	27
15.2 MapReduce Scripts	27
15.2.1 Example — TRANSFORM with a Python Script	28
15.2.2 Example — MAP and REDUCE with Nested Query	28
16 Joins	29
16.1 Inner Join	29
16.2 Outer Joins	30
16.3 Semi Joins	30
16.4 Map Joins	31
17 Subqueries	31
18 Views	32
19 User-Defined Functions (UDFs)	33
19.1 Three Types of UDFs	33
19.2 UDTF Example — explode()	33

1. Running Hive

Hive is a data warehouse system built on top of Hadoop that allows SQL-like querying of large datasets. This section covers the practical aspects of running and configuring Hive in a real cluster environment.

1.1. Configuring Hive

Hive uses an **XML configuration file** similar to Hadoop's configuration system.

1.1.1. Key Configuration Files

File	Purpose
hive-site.xml	Main config file; user-defined properties set here persist every run. Located in Hive's <code>conf/</code> directory.
hive-default.xml	Documents all properties Hive exposes along with their default values.

1.1.2. Overriding the Configuration Directory

You can override the default config directory using the `--config` option:

```
% hive --config /Users/tom/dev/hive-conf
```

This specifies the **directory**, not the file itself. Useful when switching between multiple cluster configurations. Alternatively, set the environment variable: `HIVE_CONF_DIR=/path/to/conf`

1.1.3. Cluster Connection Details in hive-site.xml

`hive-site.xml` is the standard place to configure cluster connectivity:

- `fs.defaultFS` — specifies the filesystem (e.g., HDFS)
- `yarn.resourcemanager.address` — specifies the YARN resource manager
- If **not set**, defaults to *local filesystem* and *local job runner* — useful for testing on small datasets

1.1.4. Per-Session Configuration using `-hiveconf`

Properties can be set for a single session without editing any file:

```
% hive -hiveconf fs.defaultFS=hdfs://localhost \
-hiveconf mapreduce.framework.name=yarn \
-hiveconf yarn.resourcemanager.address=localhost:8032
```

1.1.5. Setting Up Shared Directories (Multi-User Setup)

When multiple users share a Hive cluster, shared HDFS directories must be writable:

```
% hadoop fs -mkdir /tmp
% hadoop fs -chmod a+w /tmp
% hadoop fs -mkdir -p /user/hive/warehouse
% hadoop fs -chmod a+w /user/hive/warehouse
```

Tip: If all users belong to the same Unix group, **g+w** permissions are sufficient on the warehouse directory instead of **a+w**.

1.1.6. Changing Settings Within a Session — SET Command

The **SET** command lets you change Hive settings *inside* a running session:

```
hive> SET hive.enforce.bucketing=true;    -- Set a property
hive> SET hive.enforce.bucketing;         -- View a property
hive> SET;                                -- List all Hive-set
      properties
hive> SET -v;                             -- List ALL properties
      (incl. Hadoop defaults)
```

1.1.7. Configuration Precedence Hierarchy

Properties are resolved in this priority order (**lower number = higher priority**):

1. **Hive SET command** (highest priority — runtime override)
2. **Command-line -hiveconf option**
3. **hive-site.xml** and Hadoop site files (**core-site.xml**, **hdfs-site.xml**, **mapred-site.xml**, **yarn-site.xml**)
4. **Hive defaults** and Hadoop default files (**core-default.xml**, etc.) (lowest priority)

Exam Tip: The **SET** command always overrides everything else. Default files have the lowest priority and are overridden by everything above them.

1.2. Execution Engines

Hive supports multiple execution engines that process its queries on the cluster.

Engine	Description
MapReduce (mr)	Default engine. Intermediate output materialized to HDFS — higher I/O overhead.
Apache Tez	DAG-based engine; writes intermediate output to local disk (avoiding HDFS replication overhead). More flexible & faster.
Apache Spark	DAG-based; can store intermediate output in memory . Support was in progress.

The execution engine is controlled via the property `hive.execution.engine` (default: `mr`). Switch engines per query:

```
hive> SET hive.execution.engine=tez;
```

Key Advantage of Tez/Spark over MapReduce: Both are DAG engines that avoid replication overhead by not writing intermediate results to HDFS — Tez writes to local disk; Spark can keep results in memory.

1.3. Logging

- **Log location:** `${java.io.tmpdir}/${user.name}/hive.log` (usually `/tmp/${user.name}/hive.log`).
- Useful for diagnosing **configuration problems** and errors
- **Config file:** `conf/hive-log4j.properties` — edit to change log levels

Override log directory for a session:

```
% hive -hiveconf hive.log.dir='/tmp/${user.name}'
```

Send debug messages to console:

```
% hive -hiveconf hive.root.logger=DEBUG,console
```

1.4. Hive Services

Hive can run several services, specified via `hive --service <name>`.

Service	Description
<code>cli</code>	Command-line interface (shell). Default service.
<code>hiveserver2</code>	Runs Hive as a Thrift server. Supports authentication and multi-user concurrency . Required for Thrift, JDBC, ODBC clients. Default port: 10000 .
<code>beeline</code>	CLI that works in embedded mode or connects to HiveServer2 via JDBC.
<code>hwi</code>	Hive Web Interface — browser-based alternative to CLI; no client install needed.
<code>jar</code>	Hive equivalent of <code>hadoop jar</code> ; runs Java apps with Hadoop+Hive on classpath.
<code>metastore</code>	Runs metastore as a standalone remote process. Default port: 9083 .

hiveserver2 is important — it enables **multi-language** client access via Thrift, and supports **concurrent multi-user** access, unlike the original HiveServer.

1.5. Hive Clients

When Hive runs as a server (`hive --service hiveserver2`), clients connect via:

1.5.1. Thrift Client

- Hive server is exposed as a **Thrift service**
- Any language supporting Thrift can connect (Python, Ruby, Java, etc.)

1.5.2. JDBC Driver

- Type 4 (pure Java) JDBC driver
- Class: `org.apache.hadoop.hive.jdbc.HiveDriver`
- **Remote mode URI:** `jdbc:hive2://host:port/dbname` — connects to a separate HiveServer2 process
- **Embedded mode URI:** `jdbc:hive2://` — Hive runs in the *same JVM* as the app; no Thrift server needed
- Beeline CLI uses JDBC to communicate with Hive

1.5.3. ODBC Driver

- Allows BI tools and ODBC-supporting applications to connect to Hive
- **Not shipped** with Apache Hive — available from third-party vendors
- Uses Thrift internally to communicate with the Hive server

Remember: JDBC = pure Java (Type 4), ODBC = for BI tools, Thrift = language-agnostic. All three ultimately use Thrift to talk to HiveServer2.

2. The Metastore

The **metastore** is the *central repository of Hive metadata*. It is divided into two pieces:

- **A service** — handles metadata operations
- **A backing store** — the actual database storing the metadata

2.1. Hive Architecture

2.2. Metastore Configurations

There are three metastore configurations, each suited to different use cases:

2.2.1. 1. Embedded Metastore (Default)

- Metastore service runs in the **same JVM** as the Hive service
- Uses an **embedded Derby database** backed by local disk
- Simple to set up — good for getting started

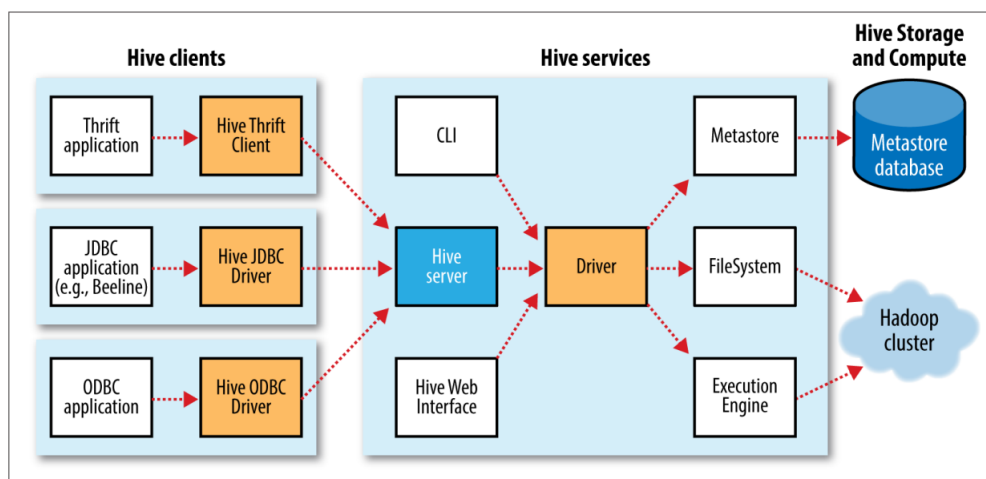


Figure 17-1. Hive architecture

Figure 1: Hive Architecture

- **Limitation:** Only **one Hive session** can access the metastore at a time (Derby allows only one connection to disk files at a time)
- Starting a second session throws an error

2.2.2. 2. Local Metastore

- Metastore service still runs in the **same process** as Hive
- But connects to a **database in a separate process** (same machine or remote machine)
- Any **JDBC-compliant database** can be used
- Configured via `javax.jdo.option.*` properties
- **MySQL** is the most popular choice — set:

```
javax.jdo.option.ConnectionURL =
    jdbc:mysql://host/dbname?createDatabaseIfNotExist=true

javax.jdo.option.ConnectionDriverName =
    com.mysql.jdbc.Driver
```

Place the MySQL Connector/J JAR file in Hive's `lib/` directory to add it to the classpath automatically.

2.2.3. 3. Remote Metastore

- One or more metastore servers run in **completely separate processes** from the Hive service
- **Better manageability and security** — database tier can be firewalled off

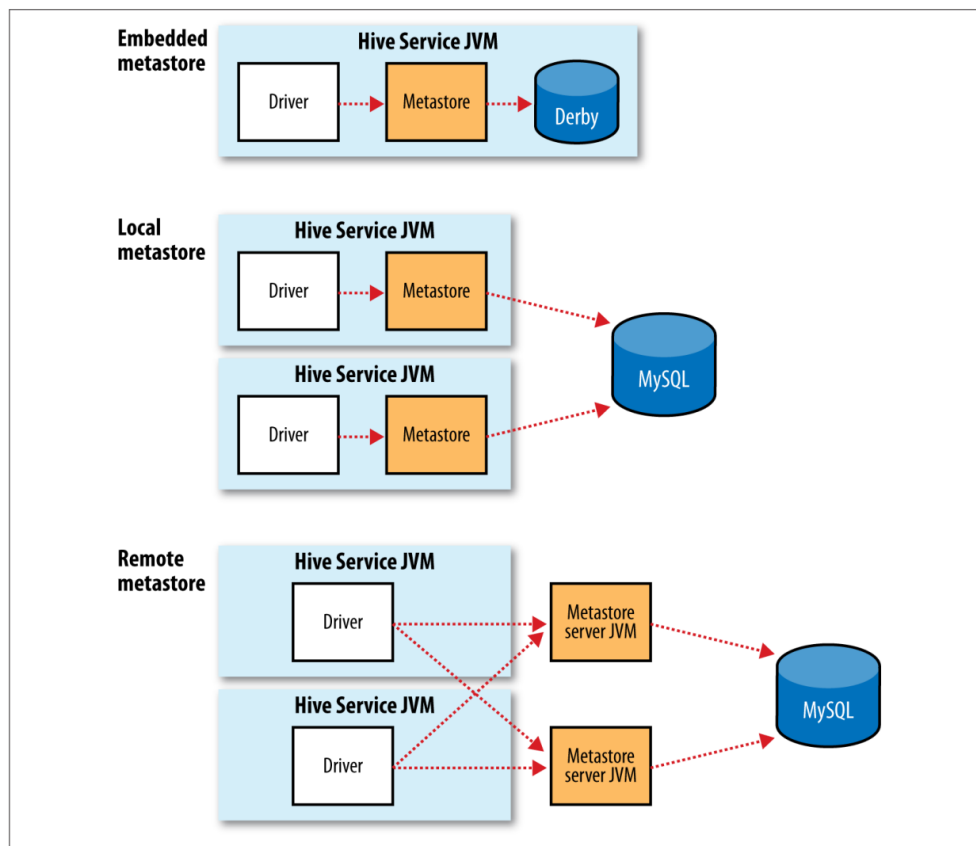


Figure 17-2. Metastore configurations

Figure 2: Metastore Configurations

- Clients no longer need database credentials
- Configure by setting `hive.metastore.uris` to the metastore server URI(s):

```
hive.metastore.uris = thrift://host:port
```

Multiple URIs can be comma-separated — clients connect in **round-robin** fashion.

Exam Summary — Metastore Types:

Embedded → same JVM, Derby DB, single session only

Local → same JVM, external JDBC DB (e.g. MySQL), multi-session

Remote → separate process, Thrift URI, most secure & scalable

2.3. Important Metastore Configuration Properties

Property	Description
<code>hive.metastore.warehouse.dir</code>	Directory (relative to <code>fs.defaultFS</code>) where managed tables are stored. Default: <code>/user/hive/warehouse</code>
<code>hive.metastore.uris</code>	Remote metastore URIs (comma-separated). If not set, uses in-process metastore.
<code>javax.jdo.option.ConnectionURL</code>	JDBC URL of the metastore database.
<code>javax.jdo.option.ConnectionDriverName</code>	JDBC driver classname.
<code>javax.jdo.option.ConnectionUserName</code>	JDBC username. Default: <code>APP</code>
<code>javax.jdo.option.ConnectionPassword</code>	JDBC password. Default: <code>mine</code>

3. Comparison with Traditional Databases

Although Hive supports a SQL-like interface, its HDFS and MapReduce foundations create key architectural differences from traditional RDBMS systems.

3.1. Schema on Read vs. Schema on Write

	Traditional DB (Schema on Write)	Hive (Schema on Read)
When is schema enforced?	At data load time	At query time
Invalid data	Rejected on load	Not caught until query
Load speed	Slower (parse + serialize)	Very fast (just file copy/-move)
Query speed	Faster (indexes + compression)	Slower (no pre-indexing)
Flexibility	Less flexible	More flexible (multiple schemas on same data possible)

Key Insight: Schema on Read = load fast, query slower. Schema on Write = load slower, query faster. Hive shines when the schema is **not known at load time**.

3.2. Updates, Transactions, and Indexes

3.2.1. Updates and Inserts

- Traditional Hive approach: full-table scans, update = transform into a new table

- **INSERT INTO** — adds new data files in bulk to an existing table (long supported)
- **From release 0.14.0:**
 - **INSERT INTO TABLE...VALUES** — insert small batches of SQL-computed values
 - **UPDATE** and **DELETE** rows are now supported
- **HDFS limitation:** No in-place file updates, so changes are stored as **small delta files**
- Delta files are periodically **merged into base table files** by background MapReduce jobs run by the metastore

3.2.2. Transactions

- Introduced in **Hive 0.13.0**
- **INSERT**, **UPDATE**, **DELETE** only work within a **transactions-enabled table**
- Queries reading the table are guaranteed to see a **consistent snapshot**

3.2.3. Locking

- Hive supports **table-level and partition-level locking**
- Prevents conflicts (e.g., one process dropping a table while another reads it)
- Locks are managed **transparently via ZooKeeper** — users don't acquire/release manually
- View lock info using: **SHOW LOCKS**
- **Disabled by default**

3.2.4. Indexes

- Indexes can speed up queries that filter on specific columns
- Example: **SELECT * FROM t WHERE x = a** benefits from an index on **x**
- **Two index types:**

Index Type	Description
Compact	Stores HDFS block numbers of each value (not file offsets). Low disk usage. Effective when values are clustered in nearby rows.
Bitmap	Uses compressed bitsets to store rows where a value appears. Best for low-cardinality columns (e.g., gender, country).

Index implementation is pluggable — designed so different implementations can be added for different use cases.

4. SQL-on-Hadoop Alternatives

Since Hive's creation, several other SQL-on-Hadoop engines have emerged to address its limitations, particularly around query performance.

4.1. Cloudera Impala

- Open source **interactive SQL engine** — one of the first Hive alternatives
- Gives an **order of magnitude performance boost** over Hive on MapReduce
- Uses a **dedicated daemon** running on each datanode in the cluster
- **Query execution flow:**
 - Client contacts any node running an Impala daemon
 - That node becomes the **coordinator node**
 - Coordinator sends work to other Impala daemons and combines results
- Uses the **Hive metastore**; supports Hive formats and most HiveQL + SQL-92
- Easy to migrate between Impala and Hive, or run both on the same cluster

4.2. Hive's Response — Stinger Initiative

- Launched by **Hortonworks** to improve Hive performance
- Key improvements:
 - Support for **Tez** as an execution engine
 - Addition of a **vectorized query engine**

4.3. Other SQL-on-Hadoop Alternatives

Engine	Key Details
Presto	From Facebook. Similar architecture to Impala.
Apache Drill	Similar to Impala. Targets SQL:2011 (not HiveQL).
Spark SQL	Uses Spark as underlying engine. Lets you embed SQL queries in Spark programs. Offers some Hive compatibility.
Apache Phoenix	SQL on HBase . JDBC driver turns queries into HBase scans. Uses HBase coprocessors for server-side aggregation. Metadata stored in HBase.

Spark SQL vs. Hive on Spark:

Hive on Spark = Spark used as execution engine *inside* Hive — has all Hive features.

Spark SQL = a new SQL engine with *some* Hive compatibility — not part of Hive project.

5. HiveQL

Hive's SQL dialect is called **HiveQL**. It is a mixture of:

- **SQL-92** (level of support has improved over time)
- **MySQL** dialect
- **Oracle SQL** dialect
- Features from **SQL:2003** — e.g., window/analytic functions
- Non-standard extensions inspired by MapReduce — e.g., multitable inserts, **TRANSFORM**, **MAP**, **REDUCE** clauses

5.1. SQL vs. HiveQL — Feature Comparison

Feature	SQL	HiveQL
Updates	UPDATE, INSERT, DELETE	UPDATE, INSERT, DELETE (limited support)
Transactions	Supported	Limited support
Indexes	Supported	Supported
Data Types	Integral, float, fixed-point, text, binary, temporal	All SQL types + Boolean, Array, Map, Struct
Functions	Hundreds built-in	Hundreds built-in
Multitable Inserts	Not supported	Supported
CREATE TABLE...AS SELECT	Not valid SQL-92	Supported
SELECT	SQL-92	SQL-92 + SORT BY (partial order) + LIMIT
Joins	SQL-92 variants	Inner, outer, semi, map, cross joins
Subqueries	Any clause	FROM, WHERE, HAVING only (uncorrelated not supported)
Views	Updatable (materialized or not)	Read-only (no materialized views)
Extension Points	UDFs, stored procedures	UDFs, MapReduce scripts

6. Data Types

Hive supports both **primitive** and **complex** data types.

6.1. Primitive Data Types

Category	Type	Description	Literal Example
Boolean	BOOLEAN	True/false value	TRUE
4*Integer	TINYINT	1-byte signed integer (128 to 127)	1Y
	SMALLINT	2-byte signed integer (32,768 to 32,767)	1S
	INT	4-byte signed integer	1
	BIGINT	8-byte signed integer	1L
3*Floating	FLOAT	4-byte single-precision float	1.0
	DOUBLE	8-byte double-precision float	1.0
	DECIMAL	Arbitrary-precision signed decimal	1.0
3*Text	STRING	Unbounded variable-length string (max 2 GB)	'a'
	VARCHAR	Variable-length string (1–65535 chars)	'a'
	CHAR	Fixed-length, space-padded string	'a'
Binary	BINARY	Variable-length byte array	Not supported
2*Temporal	TIMESTAMP	Nanosecond-precision timestamp	'2012-01-02 03:04:05'
	DATE	Date (year, month, day)	'2012-01-02'

6.1.1. Key Notes on Primitive Types

- Integer types map to Java primitives: TINYINT=byte, SMALLINT=short, INT=int, BIGINT=long
- **DECIMAL(precision, scale)**: e.g., DECIMAL(5,2) stores 999.99 to 999.99
 - Default scale = 0, default precision = 10, max precision = 38
 - Scale must be $\leq$ precision
- **STRING** max theoretical size = 2 GB
- **CHAR** trailing spaces are *ignored* in string comparisons
- **TIMESTAMP** has no built-in time zone; use `to_utc_timestamp` / `from_utc_timestamp` for conversions

6.2. Complex Data Types

Type	Description	Literal Example
ARRAY	Ordered collection of same-type fields	<code>array(1, 2)</code>
MAP	Unordered key-value pairs. Keys must be primitives; values any type. Same-type keys and values per map.	<code>map('a', 1, 'b', 2)</code>
STRUCT	Named fields of different types (like a record)	<code>struct('a', 1, 1.0)</code>
UNION	Value that matches exactly one of defined types; tagged with a zero-indexed integer	<code>create_union(1, 'a', 63)</code>

6.2.1. Complex Type Declaration (Angled Bracket Notation)

```
CREATE TABLE complex (
  c1 ARRAY<INT>,
  c2 MAP<STRING, INT>,
  c3 STRUCT<a:STRING, b:INT, c:DOUBLE>,
  c4 UNIONTYPE<STRING, INT>
);
```

6.2.2. Field Accessor Operators

```
hive> SELECT c1[0], c2['b'], c3.c, c4 FROM complex;
-- Output: 1      2      1.0      {1:63}
```

Type	Accessor
ARRAY	<code>array_col[index]</code> — zero-based index
MAP	<code>map_col['key']</code> — key lookup
STRUCT	<code>struct_col.field_name</code> — dot notation
UNION	Returns <code>{tag:value}</code> format

Exam Tip: Complex types support **arbitrary nesting**. `array`, `map`, `struct`, and `create_union` are **built-in Hive functions** used as literals.

7. Operators and Functions

7.1. Operators

Hive provides the standard SQL operators, following MySQL conventions:

Operator Type	Examples
Relational	<code>x = 'a'</code> (equality), <code>x IS NULL</code> (nullity), <code>x LIKE 'a%'</code> (pattern match)
Arithmetic	<code>x + 1</code> (addition), <code>x * 2</code> (multiplication), etc.
Logical	<code>x OR y</code> , <code>x AND y</code> , <code>NOT x</code>

MySQL deviation from SQL-92: `||` means **logical OR** (not string concatenation). Use the `concat()` function for string concatenation in both MySQL and Hive.

7.2. Built-in Functions

Hive has a large library of built-in functions across categories:

- **Mathematical & Statistical** — numeric operations, aggregations
- **String functions** — manipulation, formatting
- **Date functions** — operations on string representations of dates
- **Conditional functions** — IF, CASE, COALESCE, etc.
- **Aggregate functions** — SUM, COUNT, AVG, etc.
- **XML functions** — using `xpath`
- **JSON functions** — parsing JSON data

```
hive> SHOW FUNCTIONS;           -- List all functions
hive> DESCRIBE FUNCTION length; -- Usage info for a
    function
-- Output: length(str | binary) - Returns the length of str
--                or number of bytes in binary data
```

If no built-in function meets your need, you can write a **User-Defined Function (UDF)**.

7.3. Type Conversions

Hive's primitive types form a **hierarchy** that governs implicit conversions:

- Any **numeric type** can be implicitly converted to a **wider numeric type** or to a **text type** (STRING, VARCHAR, CHAR)
- All **text types** can be implicitly converted to another text type, or to **DOUBLE** or **DECIMAL**
- **BOOLEAN** cannot be converted to any other type (implicitly or otherwise)
- **TIMESTAMP** and **DATE** can be implicitly converted to a text type
- **Narrowing** conversions (e.g., INT → TINYINT) require explicit CAST

Explicit conversion using CAST:


```
CAST('1' AS INT)    -- Returns integer 1
CAST('X' AS INT)    -- Returns NULL (cast failure)
```

Exam Tip: Failed CAST returns **NULL**, not an error. Implicit widening is allowed; implicit narrowing is not.

8. Tables

A Hive table consists of two parts:

- **Data** — typically stored in HDFS (can also be local FS or S3)
- **Metadata** — stored in a relational database via the Metastore

8.1. Multiple Database / Schema Support

```
CREATE DATABASE dbname;    -- Create a new database
USE dbname;                -- Switch to database
DROP DATABASE dbname;      -- Delete database
-- Fully qualify a table:
SELECT * FROM dbname.tablename;
```

If no database is specified, tables belong to the **default** database.

8.2. Managed Tables vs. External Tables

	Managed Table	External Table
Also called	Internal table	External table
Data location	Moved into Hive warehouse directory	Stays at existing location outside warehouse
LOAD behaviour	Moves data into warehouse	Does not move data
DROP behaviour	Deletes both data and metadata	Deletes only metadata ; data untouched
Location check	Implicit (warehouse)	Not checked at creation time
Use when	All processing done in Hive	Sharing data with other tools

8.2.1. Managed Table Example

```
CREATE TABLE managed_table (dummy STRING);
LOAD DATA INPATH '/user/tom/data.txt' INTO TABLE managed_table;
-- Moves data to: hdfs://user/hive/warehouse/managed_table/
```

```
DROP TABLE managed_table;
-- Deletes BOTH metadata AND data permanently
```

8.2.2. External Table Example

```
CREATE EXTERNAL TABLE external_table (dummy STRING)
  LOCATION '/user/tom/external_table';
LOAD DATA INPATH '/user/tom/data.txt' INTO TABLE external_table
  ;

DROP TABLE external_table;
-- Deletes ONLY metadata; data at LOCATION is preserved
```

Common Pattern: Use an **external table** to access raw HDFS data created by another process, then use a Hive transform to move it into a **managed table** for processing. External tables are also used to **export data** from Hive for other applications.

Key: LOAD is a fast **move/rename** operation — Hive does **not** validate data against schema on load. Mismatches only appear at query time (often as NULLs).

9. Partitions and Buckets

9.1. Partitions

Partitioning divides a table into **coarse-grained parts** based on the value of a partition column (e.g., date, country). This allows queries on specific slices of data to scan **only the relevant partitions** — much faster than full-table scans.

- A table can be partitioned on **multiple dimensions**
- Partition columns are **not stored in the data files** — derived from directory names
- Hive performs **input pruning** — only scans relevant partitions during a query

9.1.1. Creating and Loading a Partitioned Table

```
-- Create partitioned table
CREATE TABLE logs (ts BIGINT, line STRING)
PARTITIONED BY (dt STRING, country STRING);

-- Load data into a specific partition
LOAD DATA LOCAL INPATH 'input/hive/partitions/file1'
INTO TABLE logs
PARTITION (dt='2001-01-01', country='GB');
```

9.1.2. Partition Directory Structure (HDFS)

At the filesystem level, partitions are **nested subdirectories** of the table directory:

```
/user/hive/warehouse/logs
  dt=2001-01-01/
    country=GB/
      file1
      file2
    country=US/
      file3
  dt=2001-01-02/
    country=GB/
      file4
    country=US/
      file5
      file6
```

9.1.3. Querying Partitions

```
-- View all partitions
hive> SHOW PARTITIONS logs;
-- Output:
-- dt=2001-01-01/country=GB
-- dt=2001-01-01/country=US
-- dt=2001-01-02/country=GB
-- dt=2001-01-02/country=US

-- Query with partition filter (input pruning kicks in)
SELECT ts, dt, line
FROM logs
WHERE country='GB';
-- Only scans: file1, file2, file4 (GB partitions only)
```

Exam Tip: Partition columns appear in **SELECT** results but are **not stored in data files** — Hive reads them from directory names. Partitioning does **not** prevent querying across all partitions when needed.

9.2. Buckets

Buckets subdivide partitions (or tables) into **fixed, fine-grained segments** for more efficient querying, especially for **sampling** and **join optimization**.

- Example: bucketing by `user_id` allows fast user-based queries on a **randomized sample** of users
- Provides extra structure beyond partitions
- Used for efficient **map-side joins** and **sampling queries**

10. Buckets

Bucketing subdivides tables or partitions into **fixed, fine-grained segments** for two main reasons:

1. **More efficient queries** — especially **map-side joins** on bucketed columns
2. **More efficient sampling** — only relevant buckets are read, not the whole table

10.1. Creating a Bucketed Table

Use the `CLUSTERED BY` clause to specify bucketing column(s) and number of buckets:

```
-- Basic bucketed table
CREATE TABLE bucketed_users (id INT, name STRING)
CLUSTERED BY (id) INTO 4 BUCKETS;

-- Bucketed + sorted (enables efficient merge-sort map-side
  join)
CREATE TABLE bucketed_users (id INT, name STRING)
CLUSTERED BY (id) SORTED BY (id ASC) INTO 4 BUCKETS;
```

Hive assigns rows to buckets by **hashing the bucket column value** and taking **modulo the number of buckets**. Each bucket is physically a **separate file** in the table/partition directory.

10.2. Populating a Bucketed Table

```
-- Enable bucketing enforcement
hive> SET hive.enforce.bucketing=true;

-- Insert from existing table
INSERT OVERWRITE TABLE bucketed_users
SELECT * FROM users;
```

After insertion, 4 files are created (one per bucket):

```
000000_0    -- Bucket 1: users with id % 4 == 0 (e.g., id=0,
             id=4)
000001_0    -- Bucket 2: users with id % 4 == 1
000002_0    -- Bucket 3: users with id % 4 == 2
000003_0    -- Bucket 4: users with id % 4 == 3
```

Warning: Hive does **not** check that data files on disk are consistent with the bucket definition. A mismatch causes errors or undefined behavior at query time. Always let Hive perform the bucketing via `INSERT`.

10.3. Sampling with TABLESAMPLE

```
-- Sample bucket 1 out of 4 (returns ~25% of data)
SELECT * FROM bucketed_users
TABLESAMPLE(BUCKET 1 OUT OF 4 ON id);
-- Output: 4 Ann, 0 Nat
```

```
-- Sample bucket 1 out of 2 (returns ~50% of data)
SELECT * FROM bucketed_users
TABLESAMPLE(BUCKET 1 OUT OF 2 ON id);
-- Output: 4 Ann, 0 Nat, 2 Joe
```

```
-- Sampling a NON-bucketed table (inefficient -- scans entire
  dataset)
SELECT * FROM users
TABLESAMPLE(BUCKET 1 OUT OF 4 ON rand());
```

	Bucketed Table	Non-Bucketed Table
Sampling	Reads only matching buckets	Scans entire dataset
Map-side Join	Efficient (bucket-to-bucket)	Not possible
Data organisation	Hash-partitioned files	Single set of files

Map-side Join rule: Two tables bucketed on the **same columns** with the **same (or multiple) number of buckets** can use map-side joins. A mapper processing one bucket of the left table only fetches the **corresponding bucket** from the right table. Sorted buckets allow an even more efficient **merge-sort join**. Bucket numbering is **1-based** in TABLESAMPLE.

11. Storage Formats

Table storage in Hive has **two dimensions**:

- **Row format** — how rows and fields are stored (defined by a **SerDe**)
- **File format** — the container format for fields in a row

11.1. SerDe (Serializer-Deserializer)

- **Deserializer** — used at **query time**: converts raw bytes from file into Hive's internal row objects
- **Serializer** — used at **INSERT/CTAS time**: converts Hive's internal row representation into bytes written to the output file

11.2. Default Storage Format — Delimited Text

When no `ROW FORMAT` or `STORED AS` clause is specified, Hive uses **delimited text** with one row per line.

Delimiter	ASCII Code	Used For
Field delimiter	\001 (Ctrl-A)	Separates fields/columns in a row
Collection delimiter	\002 (Ctrl-B)	Separates items in ARRAY, STRUCT, or MAP key-value pairs
Map key delimiter	\003 (Ctrl-C)	Separates key and value in a MAP
Line delimiter	\n (newline)	Separates rows

The explicit equivalent of the default `CREATE TABLE`:

```
CREATE TABLE ...
ROW FORMAT DELIMITED
  FIELDS TERMINATED BY '\001'           -- Ctrl-A
  COLLECTION ITEMS TERMINATED BY '\002' -- Ctrl-B
  MAP KEYS TERMINATED BY '\003'         -- Ctrl-C
  LINES TERMINATED BY '\n'
STORED AS TEXTFILE;
```

- Hive internally uses **LazySimpleSerDe** for delimited text
- **Lazy** = fields are deserialized only when accessed
- **Not compact** — values stored as verbose text (e.g., `true/false` as strings)
- Hive supports **8 levels of delimiters** (ASCII codes 1–8); only the **first 3** can be overridden

Why Ctrl-A and not Tab? Ctrl-A is less likely to appear in actual field data. There is **no escape mechanism** for delimiter characters in Hive, so choose delimiters that won't appear in data.

11.3. Binary Storage Formats

Binary formats are specified via the `STORED AS` clause. They fall into two categories:

Category	Best Used When
Row-oriented	Queries need many columns of a single row at once
Column-oriented	Queries access only a small number of columns

Format	Orientation	Key Features
Sequence File	Row	General-purpose, splittable, compressible
Avro	Row	General-purpose, splittable, compressible + schema evolution + multiple language bindings
Parquet	Column	Native Hive support; efficient for column-selective queries
RCFile	Column	Native Hive support
ORCFile	Column	Native Hive support; optimized RC format

Usage examples:

```
-- Avro format with Snappy compression
SET hive.exec.compress.output=true;
SET avro.output.codec=snappy;
CREATE TABLE ... STORED AS AVRO;

-- Sequence file
CREATE TABLE ... STORED AS SEQUENCEFILE;

-- Parquet (using CTAS)
CREATE TABLE users_parquet STORED AS PARQUET
AS SELECT * FROM users;
```

11.4. Custom SerDe — RegexSerDe

Used when data has a **fixed-width** or **regex-parseable** format:

```
CREATE TABLE stations (usaf STRING, wban STRING, name STRING)
ROW FORMAT SERDE 'org.apache.hadoop.hive.contrib.serde2.
  RegexSerDe'
WITH SERDEPROPERTIES (
  "input.regex" = "(\d{6}) (\d{5}) (.{29}) .*"
);

-- Load data
LOAD DATA LOCAL INPATH "input/ncdc/metadata/stations-fixed-
  width.txt"
INTO TABLE stations;

-- Query
hive> SELECT * FROM stations LIMIT 4;
```

- SERDE keyword + fully qualified Java class name specifies the SerDe
- WITH SERDEPROPERTIES passes config to the SerDe
- input.regex — Java regex; **capturing groups** become columns

- SerDe is **not invoked during LOAD** — only during reads/writes

RegexSerDe is inefficient for general storage. Use it to **import** data into Hive, then copy to a binary format (Parquet, Avro, ORC) for production use.

11.5. Storage Handlers

For storage systems Hive cannot access natively (e.g., **HBase**):

```
CREATE TABLE ... STORED BY 'storage.handler.class.Name'
```

- Uses **STORED BY** clause instead of **ROW FORMAT + STORED AS**
- Primary use case: **HBase integration**

12. Importing Data

Data can be brought into Hive tables in three main ways:

1. **LOAD DATA** — copy/move files directly into the table directory
2. **INSERT** statement — populate from another Hive table
3. **CTAS** (CREATE TABLE...AS SELECT) — create a new table from query results

To import data **directly from a relational database** into Hive, use **Sqoop**.

12.1. INSERT Statement

```
-- Basic insert (overwrites target table)
INSERT OVERWRITE TABLE target
SELECT col1, col2
  FROM source;

-- Insert into a specific partition
INSERT OVERWRITE TABLE target
PARTITION (dt='2001-01-01')
SELECT col1, col2
  FROM source;

-- Dynamic partition insert (partition value from SELECT)
INSERT OVERWRITE TABLE target
PARTITION (dt)
SELECT col1, col2, dt
  FROM source;

-- Append records (non-destructive)
INSERT INTO TABLE target
```



```
SELECT col1, col2
  FROM source;

-- Insert literal values (from Hive 0.14.0)
INSERT INTO TABLE target VALUES (val1, val2);
```

OVERWRITE replaces existing data in the table/partition. **INTO** appends to existing data. **Dynamic partition insert** determines partition value from the **SELECT** query itself.

12.2. Multitable Insert

In HiveQL, the **INSERT** can be turned around to start with **FROM**, enabling **multiple INSERT clauses in a single query**. The source table is scanned **only once** — more efficient than separate **INSERT** statements.

```
FROM records2
INSERT OVERWRITE TABLE stations_by_year
  SELECT year, COUNT(DISTINCT station)
  GROUP BY year
INSERT OVERWRITE TABLE records_by_year
  SELECT year, COUNT(1)
  GROUP BY year
INSERT OVERWRITE TABLE good_records_by_year
  SELECT year, COUNT(1)
  WHERE temperature != 9999 AND quality IN (0, 1, 4, 5, 9)
  GROUP BY year;
```

Multitable insert scans the source table **once** and writes to **three** different output tables simultaneously — a major efficiency gain for large datasets.

12.3. CREATE TABLE...AS SELECT (CTAS)

Creates a new table and populates it with the results of a **SELECT** query in one step. Column definitions are **derived automatically** from the **SELECT** clause.

```
CREATE TABLE target
AS
SELECT col1, col2
FROM source;
```

- The new table's column names and types match the **SELECT** output
- **Atomic operation** — if the **SELECT** fails, the table is **not created**
- Useful when query output is too large to dump to console, or needs further processing

13. Altering Tables

Because Hive uses **schema-on-read**, table definitions can be changed after creation. However, it is the user's responsibility to ensure the underlying data reflects any structural changes.

13.1. Rename a Table

```
ALTER TABLE source RENAME TO target;
```

- Updates table metadata
- **Managed table:** also renames the underlying HDFS directory (/user/hive/warehouse/source → /user/hive/warehouse/target)
- **External table:** only metadata is updated; directory is **not moved**

13.2. Add a New Column

```
ALTER TABLE target ADD COLUMNS (col3 STRING);
```

- New column is added **after existing non-partition columns**
- Existing data files are **not updated** — queries return **NULL** for the new column
- Common practice: create a new table with new columns and populate via SELECT

13.3. Changing Column Metadata

- Column name or data type can be changed via ALTER TABLE
- Works cleanly if the old data type can be **interpreted as the new type**

14. Dropping Tables

Command	Behaviour
DROP TABLE t	Deletes both data and metadata for managed tables. For external tables, deletes only metadata ; data is untouched.
TRUNCATE TABLE t	Deletes all data but keeps the table definition. Does not work on external tables.
dfs -rmr <dir>	Used from Hive shell to remove external table directories manually.
CREATE TABLE new LIKE old	Creates a new empty table with the same schema as an existing table.

```

DROP TABLE managed_table;      -- Deletes data + metadata
TRUNCATE TABLE my_table;       -- Keeps schema, deletes data
CREATE TABLE new_table LIKE existing_table; -- Empty copy of
                                schema

```

15. Querying Data

15.1. Sorting and Aggregating

Hive offers multiple clauses for sorting and distributing data:

Clause	Behaviour
ORDER BY	Total global sort of all input. Produces a single sorted output.
SORT BY	Sorts within each reducer — produces one sorted file per reducer. Not globally sorted.
DISTRIBUTE BY	Controls which reducer a row goes to (based on column value). Rows with same value go to same reducer.
CLUSTER BY	Shorthand for DISTRIBUTE BY + SORT BY on the same column(s) .

Example — DISTRIBUTE BY + SORT BY:

```

FROM records2
SELECT year, temperature
DISTRIBUTE BY year
SORT BY year ASC, temperature DESC;
-- Output:
-- 1949    111
-- 1949     78
-- 1950     22
-- 1950      0
-- 1950   -11

```

Exam Tip: ORDER BY = global sort (one reducer). SORT BY = per-reducer sort (multiple reducers, not globally ordered). DISTRIBUTE BY = controls which reducer gets which rows. CLUSTER BY = DISTRIBUTE BY + SORT BY on same columns.

15.2. MapReduce Scripts

Using an approach similar to **Hadoop Streaming**, Hive's TRANSFORM, MAP, and REDUCE clauses allow invoking **external scripts** from within a query.

15.2.1. Example — TRANSFORM with a Python Script

Python script (`is_good_quality.py`) to filter poor-quality weather records:

```
#!/usr/bin/env python
import re
import sys

for line in sys.stdin:
    (year, temp, q) = line.strip().split()
    if (temp != "9999" and re.match("[01459]", q)):
        print "%s\t%s" % (year, temp)
```

Using the script in Hive:

```
-- Register the script (ships it to Hadoop cluster via
Distributed Cache)
hive> ADD FILE /Users/tom/book-workspace/hadoop-book/ch17-hive/
src/main/python/is_good_quality.py;

-- Use TRANSFORM to stream data through the script
hive> FROM records2
    > SELECT TRANSFORM(year, temperature, quality)
    > USING 'is_good_quality.py'
    > AS year, temperature;
-- Output:
-- 1950      0
-- 1950     22
-- 1950    -11
-- 1949     11
-- 1949     78
```

- Fields are streamed as **tab-separated lines** to the script's stdin
- Script's tab-separated stdout is parsed back into Hive columns
- **ADD FILE** is required to register the script before use — Hive ships it to the cluster via **Distributed Cache**

15.2.2. Example — MAP and REDUCE with Nested Query

```
FROM (
    FROM records2
    MAP year, temperature, quality
    USING 'is_good_quality.py'
    AS year, temperature) map_output
REDUCE year, temperature
USING 'max_temperature_reduce.py'
AS year, temperature;
```

MAP and REDUCE keywords are equivalent to SELECT TRANSFORM. The nested form allows specifying **both a map and reduce** external script in a single query — full custom MapReduce pipeline from within HiveQL.

16. Joins

Hive simplifies join operations that are complex to implement in raw MapReduce. Hive **only supports equijoins** — join predicates must use equality (=).

16.1. Inner Join

Returns only rows where there is a **match in both tables**.

```
hive> SELECT * FROM sales;
Joe      2
Hank     4
Ali      0
Eve      3
Hank     2

hive> SELECT * FROM things;
2      Tie
4      Coat
3      Hat
1      Scarf

-- Inner join
hive> SELECT sales.*, things.*
      > FROM sales JOIN things ON (sales.id = things.id);
Joe      2      2      Tie
Hank     4      4      Coat
Eve      3      3      Hat
Hank     2      2      Tie
```

- Ali is **excluded** — her item ID (0) has no match in things
- Join on multiple columns: use AND between conditions
- Join more than two tables: add more JOIN...ON... clauses
- Alternative syntax (MySQL/Oracle style):

```
SELECT sales.*, things.*
FROM sales, things
WHERE sales.id = things.id;
```

Performance: A single join = one MapReduce job. Multiple joins on the **same column** can be done in fewer than one job per join. Use **EXPLAIN** to see the execution plan, **EXPLAIN EXTENDED** for full details.

16.2. Outer Joins

Join Type	Behaviour
LEFT JOIN	All rows from left table ; NULLs for unmatched right table columns
RIGHT JOIN	All rows from right table ; NULLs for unmatched left table columns
FULL JOIN	All rows from both tables ; NULLs where no match exists

```
-- LEFT OUTER JOIN (includes Ali with NULLs)
hive> SELECT sales.*, things.*
      > FROM sales LEFT OUTER JOIN things ON (sales.id = things.
        id);
Joe      2      2      Tie
Hank     4      4      Coat
Ali       0     NULL NULL
Eve      3      3      Hat
Hank     2      2      Tie

-- RIGHT OUTER JOIN (includes Scarf with NULLs)
hive> SELECT sales.*, things.*
      > FROM sales RIGHT OUTER JOIN things ON (sales.id = things.
        id);
Joe      2      2      Tie
Hank     2      2      Tie
Hank     4      4      Coat
Eve      3      3      Hat
NULL     NULL  1      Scarf

-- FULL OUTER JOIN (includes all rows from both tables)
hive> SELECT sales.*, things.*
      > FROM sales FULL OUTER JOIN things ON (sales.id = things.
        id);
Ali       0     NULL NULL
NULL     NULL  1      Scarf
Hank     2      2      Tie
Joe      2      2      Tie
Eve      3      3      Hat
Hank     4      4      Coat
```

16.3. Semi Joins

LEFT SEMI JOIN is Hive's efficient equivalent of an **IN** subquery. Returns rows from the **left table** that have a match in the right table.

```
-- Equivalent IN subquery
SELECT * FROM things
WHERE things.id IN (SELECT id from sales);

-- LEFT SEMI JOIN equivalent
hive> SELECT *
      > FROM things LEFT SEMI JOIN sales ON (sales.id = things.id
      );
2      Tie
4      Coat
3      Hat
```

Restriction: The right table (`sales`) may **only** appear in the `ON` clause — it **cannot** be referenced in the `SELECT` expression.

16.4. Map Joins

If one table is **small enough to fit in memory**, Hive loads it into each mapper's memory to perform the join — no reducers needed.

```
-- Map join happens automatically if one table is small
SELECT sales.*, things.*
FROM sales JOIN things ON (sales.id = things.id);
```

- **No reducers** — cannot be used for `RIGHT` or `FULL OUTER JOIN` (absence of match requires a reduce step)
- Works efficiently with **bucketed tables**: a mapper on one bucket of the left table only loads the **corresponding bucket** of the right table

```
-- Enable bucketed map join optimization
SET hive.optimize.bucketmapjoin=true;
```

Join Type Summary:

Inner — matches only. **Left Outer** — all left + NULLs for unmatched right.

Right Outer — all right + NULLs for unmatched left. **Full Outer** — all rows both sides.

Semi — left rows with a match (IN equivalent). **Map** — in-memory join, no reducer.

17. Subqueries

A **subquery** is a `SELECT` embedded inside another SQL statement. Hive has **limited** subquery support:

- Subquery in the **FROM clause** — fully supported

- Subquery in the **WHERE** clause — only for uncorrelated IN or EXISTS statements
- **Correlated subqueries** (referencing the outer query) — **not supported**

```
-- Subquery in FROM clause
-- Find mean maximum temperature per station per year
SELECT station, year, AVG(max_temperature)
FROM (
    SELECT station, year, MAX(temperature) AS max_temperature
    FROM records2
    WHERE temperature != 9999 AND quality IN (0, 1, 4, 5, 9)
    GROUP BY station, year
) mt
GROUP BY station, year;
```

- The subquery **must have an alias** (mt here) — the outer query accesses it like a table
- Subquery columns must have **unique names** for the outer query to reference them

18. Views

A **view** is a virtual table defined by a SELECT statement. Used to:

- Present simplified or aggregated data to users
- Restrict user access to specific subsets of data
- Avoid repeating complex query logic

Key: Hive views are **NOT materialized to disk** when created. The SELECT is executed only when the view is queried. Views are **read-only** — no INSERT or LOAD through a view.

```
-- Create a view for valid weather records
CREATE VIEW valid_records
AS
SELECT *
FROM records2
WHERE temperature != 9999 AND quality IN (0, 1, 4, 5, 9);

-- Create a second view based on the first
CREATE VIEW max_temperatures (station, year, max_temperature)
AS
SELECT station, year, MAX(temperature)
FROM valid_records
GROUP BY station, year;

-- Query using the view
SELECT station, year, AVG(max_temperature)
```



```
FROM max_temperatures
GROUP BY station, year;
```

- Views appear in `SHOW TABLES` output
- Inspect a view: `DESCRIBE EXTENDED view_name`
- Column names can be listed explicitly in the view definition (useful for aggregate expressions that would get auto-generated aliases)
- To **manually materialize** a view: use `CREATE TABLE...AS SELECT`
- Hive **combines view queries** into the minimum number of MapReduce jobs — it won't needlessly materialize a view at execution time

19. User-Defined Functions (UDFs)

When built-in Hive functions are insufficient, you can write **User-Defined Functions**. UDFs **must be written in Java**. For other languages, use `SELECT TRANSFORM` with external scripts.

19.1. Three Types of UDFs

Type	Input	Output	Examples
UDF	Single row	Single row	Math functions, string functions
UDAF	Multiple rows	Single row	COUNT, MAX, SUM
UDTF	Single row	Multiple rows (a table)	Table-generating functions

UDF = 1 row → 1 row **UDAF** = many rows → 1 row **UDTF** = 1 row → many rows

19.2. UDTF Example — `explode()`

```
-- Table with an ARRAY column
CREATE TABLE arrays (x ARRAY<STRING>)
ROW FORMAT DELIMITED
  FIELDS TERMINATED BY '\001'
  COLLECTION ITEMS TERMINATED BY '\002';
```

- `COLLECTION ITEMS TERMINATED BY '\002'` — array entries delimited by Ctrl-B characters
- A UDTF like `explode()` takes one row with an array and outputs **one row per element** in that array